

STM32 嵌入式系统设计

期末综合项目任务书

课程名称	STM32 嵌入式系统设计	实验项目名称	多功能嵌入式系统综合设计
实验学时	8 学时	实验类型	综合设计
实验类别	期末考核	适用专业	电子信息、自动化、物联网

一、实验目的

1. 综合运用 STM32 HAL 库、GPIO、定时器、ADC、RTC 等外设，设计完整的嵌入式应用系统。
2. 掌握状态机设计方法，实现多模式切换和复杂交互逻辑。
3. 熟练掌握非阻塞延时技术，避免使用阻塞式延时影响系统响应性。
4. 培养独立分析问题、设计方案和实现调试的能力。

二、实验原理与背景知识

2.1 状态机设计方法

嵌入式系统中的多模式切换通常采用状态机来实现。状态机由**状态**、**事件**和**转换**三要素组成：

当前状态 + 触发事件 → 下一状态 + 执行动作

本项目中，学生需要根据所选任务设计合适的状态机结构，例如：

- **任务一：**关闭 / 左转 / 右转 / 双闪 / 迎宾 / 紧急 等状态
- **任务二：**手动模式 / 报警模式 + 手动模式下的灯光状态
- **任务三：**正常运行 / 设置闹钟 / 闹钟触发 / 查询闹钟 等状态

2.2 非阻塞延时原理

传统嵌入式程序常使用 HAL_Delay() 进行延时，但该函数会**阻塞 CPU**，导致系统无法及时响应按键等外部事件。

非阻塞延时基于系统滴答定时器 (SysTick) 提供的 HAL_GetTick() 函数，该函数返回系统启动后的毫秒数。通过记录“上次执行时间”并与当前时间比较，可实现周期性执行而不阻塞主循环：

```
uint32_t last_tick = HAL_GetTick();
while (1) {
    uint32_t now = HAL_GetTick();
    if (now - last_tick >= 500) {    // 500ms 周期
        last_tick = now;
        // 执行需要定时的任务
        LED_Toggle();
    }
}
```

```

    }
    // 主循环继续执行, 可在此处检测按键
    Key_Scan();
}

```

2.3 长按与短按识别

通过记录按键**按下时刻**与**释放时刻**的时间差, 可区分短按和长按:

```

if (key_pressed) {
    press_start_tick = HAL_GetTick();
    while (key_still_pressed) {
        if (HAL_GetTick() - press_start_tick > 1000) {
            // 长按判定
            handle_long_press();
            return;
        }
    }
    // 按键释放, 时间不足 1 秒
    handle_short_press();
}

```

三、硬件说明

本实验使用 **正点原子 STM32F407 最小系统板**, 相关外设资源如下:

外设	引脚	有效电平	功能说明
LED0	PF9	低电平点亮	指示灯/左灯
LED1	PF10	低电平点亮	指示灯/右灯
KEY0	PE4	低电平有效 (按下为低)	短按/长按切换模式、设置等
KEY_UP	PA0	高电平有效 (按下为高)	紧急双闪、查询闹钟、快速增加

注意: KEY0 和 KEY_UP 的有效电平不同, 编写按键读取逻辑时需分别处理 (例如使用 HAL_GPIO_ReadPin 判断电平)。无串口通信, 所有交互通过 LED 和按键完成。

四、实验任务要求

本实验提供三个难度递进的任务, 学生**任选其一**完成。按所选任务的满分标准评分。

任务一: 多功能灯光系统 (满分 80 分)

4.1.1 功能要求

1. **短按** KEY0 循环切换模式:

关闭 → 左转 → 右转 → 双闪 → 关闭 （循环）

- **左转**: LED0 以 1Hz 闪烁 (亮 0.5 秒, 灭 0.5 秒), LED1 熄灭
- **右转**: LED1 以 1Hz 闪烁 (亮 0.5 秒, 灭 0.5 秒), LED0 熄灭
- **双闪**: LED0 和 LED1 以 2Hz 同步闪烁 (亮 0.25 秒, 灭 0.25 秒)

2. **长按 KEY0 (超过 1 秒)**: 进入 “迎宾模式”

- 判定时机: 按键按下持续时间 **>1 秒** 时立即进入 (不等松手)
- LED0 和 LED1 **交替慢闪** (0.5Hz): LED0 亮 1 秒同时 LED1 灭 → LED1 亮 1 秒同时 LED0 灭 → 循环
- **闪烁 3 次** (一次完整的交替计为 1 次, 即共执行 3 个周期的交替)
- 闪烁结束后**自动返回长按前的模式** (例如长按前是左转模式, 返回后应恢复左转闪烁状态)

3. **按下 KEY_UP**: 立即进入 “紧急双闪”

- LED0 和 LED1 以 2Hz 双闪 (亮 0.25 秒, 灭 0.25 秒同步)
- 再次按下 KEY_UP 则恢复进入紧急模式前的原模式
- 紧急模式可打断任何状态 (包括迎宾模式、设置中等), 优先级最高

4. 所有 LED 闪烁必须使用**非阻塞延时**实现, 禁止使用 HAL_Delay()。

4.1.2 评分标准 (满分 80 分)

评分项	分值	具体要求
短按循环切换模式正确	20	按键响应及时, 模式切换无误, 闪烁频率符合要求
长按迎宾模式及自动返回正确	20	长按识别准确 (>1 秒), 闪烁 3 次后恢复原模式
紧急双闪打断与恢复正确	20	KEY_UP 可随时打断, 再次按下正确恢复
非阻塞闪烁实现	10	完全不使用 HAL_Delay, 闪烁频率稳定
代码规范、注释清晰、结构合理	10	变量命名规范, 关键逻辑有注释, 状态机清晰

任务二: 温度报警与灯光联动系统 (满分 90 分)

4.2.1 功能要求

1. **温度采集**:

- 使用 STM32 内部温度传感器 (ADC1 通道 16)
- 每 **约 2 秒** 自动检测一次温度 (允许 ± 0.2 秒误差, 使用非阻塞定时实现)
- 不需要串口输出温度值

2. **手动模式** (默认模式, 温度 $< 45^{\circ}\text{C}$):

- 短按 KEY0 循环切换灯光状态:
关闭 → LED0 常亮 → LED1 常亮 → 双闪 2Hz → 关闭 （循环）
- 状态切换应通过 LED 行为直接体现

3. 自动报警模式 (温度 $\geq 45^{\circ}\text{C}$):

- 自动进入报警状态:
 - LED0 和 LED1 同时以 **2Hz 快闪**
 - 此时 **手动按键失效** (短按 KEY0 不能切换灯光, 长按 KEY0 也不做任何响应, 仅 KEY_UP 有效)
- 温度降至 45°C 以下后:
 - 自动恢复报警前的手动模式及灯光状态
 - 例如: 报警前是 “LED0 常亮”, 恢复后应为 LED0 常亮

4. 无额外健康指示 (避免与手动模式冲突): 只需实现上述功能即可。温度采集期间不应阻塞主循环。

5. 非阻塞设计:

- ADC 采集、LED 闪烁、按键检测均不能使用阻塞延时
- 温度检测周期约为 2 秒, 但不应阻塞主循环

4.2.2 评分标准 (满分 90 分)

评分项	分值	具体要求
温度采集与超限判断准确	20	ADC 配置正确, 温度计算合理, 超限判断准确
手动模式切换正常	15	常温下按键切换所有灯光状态正确
高温自动报警及 LED 快闪	20	超过 45°C 自动进入报警, LED 2Hz 快闪
温度回落后状态完全恢复	15	降温后准确恢复原模式和灯光状态
非阻塞设计	10	所有功能均使用非阻塞方式实现
代码结构、注释、状态机清晰	10	状态机设计合理, 代码可读性好

任务三: 定时迎宾灯系统 (满分 100 分)

4.3.1 功能要求 (无串口, 纯按键交互)

1. RTC 初始化与走时:

- 上电后 RTC 时间初始化为 **00:00:00**, 开始走时
- 优先使用外部 LSE 晶振 (32.768 kHz), 若硬件不支持则使用 LSI
- 闹钟设置后需要**掉电保持**, 使用 RTC 备份寄存器 (需确保 VBAT 有电) 或内部 Flash 保存

2. 设置闹钟:

- **进入设置模式:** 长按 KEY0 (>1 秒) 进入 “设置闹钟” 模式
 - LED0 慢闪 (0.5Hz) 提示进入设置模式
- **切换设置项:** 短按 KEY0 循环切换:
小时设置 → 分钟设置 → 确认保存 → 退出
 - LED0 闪烁次数表示当前项:
 - * 闪烁 **1 次** = 设置小时
 - * 闪烁 **2 次** = 设置分钟
 - * 闪烁 **3 次** = 确认保存
- **调整数值:** 短按 KEY_UP 增加当前设置项的值
 - 小时范围: 0-23 (超出回到 0)
 - 分钟范围: 0-59 (超出回到 0)

- **快速增加** (可选, 但推荐实现): 在“小时”或“分钟”项, 长按 KEY_UP 可快速增加 (建议每秒加 5)
 - **超时退出**: 若 10 秒内无任何按键操作, 自动退出设置模式, 不修改闹钟
 - **确认提示**: 确认保存后, LED0 长亮 1 秒作为提示
3. **闹钟触发**:
- 当 RTC 时间到达设定时刻时, 自动触发“迎宾模式”:
 - LED0 和 LED1 交替慢闪 (0.5Hz): LED0 亮 1 秒同时 LED1 灭 → LED1 亮 1 秒同时 LED0 灭 → 循环
 - 闪烁 **10 次** (一次完整的交替计为 1 次) 后自动停止
 - 闹钟触发期间, 按键检测正常工作 (如短按 KEY_UP 不应干扰闪烁, 可设计为允许打断)
4. **查询闹钟** (非闹钟执行期间):
- 短按 KEY_UP 可查询当前设定的闹钟时间
 - **查询方式**:
 - LED0 闪烁 **小时数** 次 (闪烁间隔 0.5 秒, 亮 0.25 秒灭 0.25 秒)
 - 停顿 1 秒
 - LED1 闪烁 **分钟数** 次 (同上间隔)
 - 例如: 闹钟设为 07:30 → LED0 闪 7 次, 停顿 1 秒, LED1 闪 30 次
5. **中断要求**:
- 必须使用 **RTC 闹钟中断** 触发迎宾模式
 - 不得在主循环中轮询 RTC 时间判断闹钟

4.3.2 评分标准 (满分 100 分)

评分项	分值	具体要求
RTC 初始化与走时正确	10	RTC 配置正确, 时间走时准确
设置闹钟交互完整可靠	25	长按进入、短按切换、KEY_UP 调整、超时退出、确认提示全部正确
闹钟时间存储与读取正确	15	闹钟设置后掉电不丢失 (使用后备寄存器或 Flash)
闹钟到时自动触发迎宾闪烁 10 次	20	RTC 闹钟中断正确配置, 到时自动触发, 闪烁 10 次
查询闹钟功能正确	15	按键查询响应, 闪烁次数表示时间正确
非阻塞设计、中断使用规范	10	所有功能非阻塞, 中断服务程序简洁
代码结构、注释、状态机清晰	5	状态机设计合理, 代码可读性好

五、通用实验要求

5.1 提交内容

每位学生需提交以下材料:

序号	内容	要求
1	Keil 工程	完整可编译的工程文件, 包含所有源文件和配置

序号	内容	要求
2	项目报告 (PDF)	含流程图、状态机图、核心代码说明、调试过程
3	演示视频	MP4 格式, 时长 ≤ 2 分钟, 展示完整功能 (任务三查询闹钟可分段加速)

5.2 代码规范

1. 文件组织:

- main.c: 主函数和主循环
- led.c / led.h: LED 控制函数
- key.c / key.h: 按键检测函数 (注意区分有效电平)
- state_machine.c / state_machine.h: 状态机实现
- 其他功能模块按需创建

2. 命名规范:

- 函数名: 模块 _ 动作, 如 LED_TurnLeft()
- 变量名: 小写下划线, 如 current_state
- 宏定义: 全大写, 如 LED_TURN_LEFT

3. 注释要求:

- 每个函数前添加功能说明注释
- 关键逻辑添加行内注释
- 状态转换需标注转换条件

5.3 非阻塞延时检查

检查点: - 代码中**不得出现** HAL_Delay() 调用 - 所有定时任务使用基于 HAL_GetTick() 的非阻塞实现 - 主循环应能在 $< 10\text{ms}$ 内完成一次迭代

六、实验报告要求

6.1 报告结构

1. 实验概述

- 1.1 实验目的
- 1.2 所选任务及理由

2. 系统设计

- 2.1 状态机设计 (含状态转换图, 标注转换条件)
- 2.2 模块划分与接口设计
- 2.3 关键算法说明

3. 核心代码说明

- 3.1 非阻塞延时实现
- 3.2 状态机实现

3.3 特殊功能实现（如长按识别、温度采集、RTC 配置等）

4. 调试与测试

4.1 调试过程记录

4.2 功能测试结果（可附视频截图）

4.3 遇到的问题及解决方案

5. 总结与心得

5.1 完成本实验的收获

5.2 有待改进之处

6.2 图表要求

1. **状态转换图**：使用 Visio、draw.io 或手绘后扫描
2. **流程图**：关键算法流程图
3. **时序图**：(可选) 多任务协作时序

七、教师评分表

学号	姓名	班级	所选任务	各项得分	总得分	百分制	备注
一/二/三				/80 /90 /100			

百分制折算公式：

最终成绩 = 实际得分 / 任务满分 × 100

例如：选任务三得 85 分 → 最终成绩 = $85/100 \times 100 = 85$ 分

八、参考资源

8.1 HAL 库参考

- HAL_GPIO_TogglePin() - GPIO 翻转
- HAL_GPIO_ReadPin() - GPIO 读取
- HAL_ADC_Start() / HAL_ADC_PollForConversion() - ADC 操作
- HAL_RTC_GetTime() / HAL_RTC_SetAlarm_IT() - RTC 操作

8.2 参考代码

- src/projects/01-turn-signal/ - 转向灯基础示例
 - src/common/drivers/ - 外设驱动参考
-

九、思考题（选做）

9.1 通用思考题（所有任务适用）

1. 如果需要增加“记忆模式”功能，即掉电重启后恢复上次的模式，应如何实现？
2. 如何使用“信号量”或“事件标志”的概念来理解本实验中的状态切换？

9.2 任务一思考题

1. 如果要求迎宾模式闪烁次数可配置（如通过长按 KEY_UP 设置闪烁次数），应如何扩展状态机？

9.3 任务二思考题

1. 如果温度在 45°C 临界点附近波动，如何避免模式频繁切换？（提示：滞回比较，如 42°C 上报、48°C 恢复）

9.4 任务三思考题

1. 如果需要设置多个闹钟（如工作日闹钟和周末闹钟），应如何扩展状态机？

说明：本任务书为开放式设计，学生可根据自身能力和兴趣选择合适难度的任务。评分标准清晰，鼓励学生在完成基本要求的基础上，进一步优化用户体验和代码质量。

最后更新：2026 年 6 月 **版本：**v2.0（适配正点原子 STM32F407 最小系统板，无串口版）